

Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación

CC3069 – Computación Paralela y Distribuida

Examen Parcial 1

Consultoría HPC usando OpenMP

Rieblur HCP

Integrantes: Abby Donis – 22440
Renato Rojas – 23813
Nery Molina – 23218

Repositorio del proyecto:

<https://github.com/Nery2004/Examen-Parcial-1-Paralela>

Guatemala, 2026

Índice

1. Resumen	3
2. Contexto y datos	3
2.1. Suma de Riemann	3
2.1.1. Propuesta secuencial	4
2.1.2. Datos de prueba	5
2.2. Blurring de imágenes	5
2.2.1. Propuesta secuencial	5
2.2.2. Estructuras de datos	6
2.2.3. Imágenes utilizadas	6
3. Estrategia de paralelización	7
3.1. Suma de Riemann	7
3.1.1. Condición de carrera	7
3.1.2. Scheduling	8
3.2. Blurring de imágenes	8
3.2.1. Balance de carga	9
4. Metodología de benchmarking	9
5. Resultados y métricas individuales	10
5.1. Nery Molina – 23218	10
5.1.1. Suma de Riemann	10
5.1.2. Blur	11
5.1.3. Evidencia de ejecución de Nery	12
5.1.4. Gráficas generadas en el benchmark de Nery	13
5.2. Renato Rojas – 23813	14
5.2.1. Suma de Riemann	14
5.2.2. Blur	14
5.2.3. Evidencia de ejecución de Renato	15
5.2.4. Gráficas generadas en el benchmark de Renato	15

5.3. Abby Donis – 22440	15
5.3.1. Suma de Riemann	15
5.3.2. Blur	16
5.3.3. Salida del benchmark de Abby	16
6. Comparación general	17
6.1. Riemann	17
6.2. Blur	17
7. Resumen de escalabilidad	18
7.1. Riemann con ocho threads	18
7.2. Blur con ocho threads	19
8. Limitaciones de las mediciones	19
9. Conclusiones	19
10. Repositorio y reproducibilidad	20
A. Gráficas adicionales	22
A.1. Nery Molina	22
A.2. Renato Rojas	23
A.3. Abby Donis	24

1. Resumen

En este trabajo se evaluaron dos problemas que originalmente se tenían en forma secuencial: la Suma de Riemann y un filtro de desenfoque o *Blur* aplicado sobre imágenes BMP. La idea principal fue comprobar si paralelizar los ciclos de mayor carga usando OpenMP realmente producía una mejora, ya que utilizar más threads no garantiza automáticamente que un programa sea más rápido.

Para la Suma de Riemann se paralelizó el recorrido de las subdivisiones y se utilizó una reducción para proteger el acumulador. En el caso del Blur se repartió el procesamiento de las filas de la imagen entre varios threads, manteniendo separadas las matrices de entrada y salida para evitar condiciones de carrera.

Las mediciones finales fueron realizadas de manera individual por Abby Donis, Renato Rojas y Nery Molina. Cada corrida probó las versiones secuencial y paralela con 1, 2, 4 y 8 threads. El script de benchmark repitió cada configuración tres veces y reportó la mediana. Los resultados mostraron una mejora clara para Riemann cuando aumentó el tamaño del problema, mientras que Blur presentó mejoras mucho más pequeñas debido a que parte importante del programa, principalmente la lectura y escritura de las imágenes, continuó siendo secuencial.

2. Contexto y datos

Los dos problemas elegidos fueron **Suma de Riemann** y **Blurring de imágenes**. Se escogieron porque en ambos existe un ciclo principal con una cantidad considerable de operaciones independientes, pero tienen un comportamiento diferente al paralelizarlos. Riemann es principalmente un problema de cálculo, mientras que Blur también depende del acceso a memoria y de operaciones de entrada y salida.

2.1. Suma de Riemann

La Suma de Riemann permite aproximar una integral definida dividiendo un intervalo en una cantidad N de partes pequeñas. En cada parte se evalúa la función y se suman las áreas aproximadas.

En el programa del proyecto se utilizó:

$$f(x) = x^2$$

en el intervalo:

$$[0, 1]$$

El valor exacto de referencia es:

$$\int_0^1 x^2 dx = \frac{1}{3}$$

La implementación utilizó la regla del punto medio. Para cada subdivisión se calculó:

$$\Delta x = \frac{b - a}{N}$$

y posteriormente:

$$x_i = a + \left(i + \frac{1}{2}\right) \Delta x$$

El resultado se obtuvo mediante:

$$I \approx \Delta x \sum_{i=0}^{N-1} f(x_i)$$

2.1.1. Propuesta secuencial

La solución secuencial recorrió todas las subdivisiones utilizando un solo ciclo. En cada iteración se calculó el punto medio, se evaluó x^2 y se acumuló el resultado en una variable de tipo `double`.

Una versión simplificada de la lógica utilizada fue:

```
double suma = 0.0;

for (long long i = 0; i < n; ++i) {
    const double x = limiteInferior
        + (static_cast<double>(i) + 0.5) * dx;

    suma += x * x;
}

double resultado = suma * dx;
```

No fue necesario guardar todos los valores de x ni de $f(x)$ dentro de arreglos. Por esta razón, el consumo extra de memoria se mantuvo aproximadamente constante:

$$O(1)$$

mientras que el trabajo computacional creció de manera lineal:

$$O(N)$$

2.1.2. Datos de prueba

En el benchmark final se utilizaron cuatro tamaños:

Cuadro 1: Valores utilizados para la Suma de Riemann.

Prueba	N
1	10,000,000
2	50,000,000
3	100,000,000
4	500,000,000

Se utilizaron tamaños grandes debido a que con valores muy pequeños la ejecución terminaba en muy poco tiempo y el costo de crear y administrar threads podía llegar a ser similar al costo del cálculo mismo. Con los tamaños utilizados fue posible observar mejor cómo cambiaba el comportamiento cuando aumentaba el trabajo.

2.2. Blurring de imágenes

El segundo problema consistió en aplicar un filtro de desenfoque de 3×3 sobre imágenes BMP. Para cada píxel interior se tomaron los nueve valores de su vecindad y se calculó un promedio para cada canal de color.

Para el canal rojo, por ejemplo:

$$R_{\text{nuevo}}(x, y) = \frac{1}{9} \sum_{j=-1}^1 \sum_{i=-1}^1 R(x+i, y+j)$$

El mismo procedimiento se realizó sobre los canales verde y azul.

Los píxeles ubicados en los bordes no fueron promediados y se copiaron directamente a la imagen de salida.

2.2.1. Propuesta secuencial

La versión secuencial recorrió primero todas las filas y columnas de la imagen. Para cada píxel interior se hicieron dos ciclos adicionales para obtener los nueve píxeles vecinos.

De forma simplificada, el procedimiento fue:

```
for (int y = 0; y < alto; y++) {  
    for (int x = 0; x < ancho; x++) {  
  
        if (es_borde) {
```

```

        copiar_pixel();
        continue;
    }

    for (int dy = -1; dy <= 1; dy++) {
        for (int dx = -1; dx <= 1; dx++) {
            sumar_vecino();
        }
    }

    calcular_promedio();
}
}

```

2.2.2. Estructuras de datos

Para trabajar con la imagen se utilizaron matrices dinámicas de `unsigned char`. Los canales de color fueron separados en:

- matriz roja de entrada;
- matriz verde de entrada;
- matriz azul de entrada;
- matriz roja de salida;
- matriz verde de salida;
- matriz azul de salida.

Separar las matrices de entrada de las matrices de salida fue importante, ya que durante el Blur todos los threads podían leer los píxeles originales sin que otro thread los estuviera modificando.

2.2.3. Imágenes utilizadas

Las imágenes se encuentran dentro de la carpeta:

`data/blur/`

El benchmark utilizó:

Cuadro 2: Archivos utilizados para las pruebas de Blur.

Archivo	Tamaño del archivo de entrada
pequena.bmp	66,614 bytes
mediana.bmp	1,049,654 bytes
grande.bmp	5,235,894 bytes

Los archivos fueron almacenados directamente en el repositorio para que los tres integrantes pudieran ejecutar exactamente el mismo conjunto de pruebas.

3. Estrategia de paralelización

3.1. Suma de Riemann

El ciclo de Riemann es un buen candidato para paralelización porque el valor de $f(x_i)$ de una iteración no depende del resultado de otra. El problema aparece al momento de acumular los resultados, ya que todos los threads necesitarían modificar la misma variable.

La directiva final utilizada fue:

```
#pragma omp parallel for reduction(+ : suma) schedule(static)
```

3.1.1. Condición de carrera

Sin una protección, varios threads podrían ejecutar al mismo tiempo una operación similar a:

```
suma += funcion(x);
```

La operación de suma no es atómica. Dos threads podrían leer el mismo valor, modificarlo y luego sobrescribir uno de los resultados.

Para evitarlo se utilizó:

```
reduction(+ : suma)
```

OpenMP mantuvo una suma parcial para cada thread y, al terminar la región paralela, combinó las sumas parciales.

Esto evitó utilizar una sección `critical` en cada iteración, lo cual habría introducido una serialización muy grande.

3.1.2. Scheduling

Se utilizaron pruebas con `static` y `dynamic`. En Riemann cada iteración realiza prácticamente las mismas operaciones, por lo que no existe un desbalance de carga considerable.

En las pruebas de optimización con $N = 100,000,000$, el scheduling estático tuvo los siguientes promedios:

Cuadro 3: Comparación de scheduling en Riemann.

Threads	Static (s)	Dynamic (s)
4	0.039558195	1.010596443
8	0.024994019	0.570872497

En este caso `dynamic` agregó mucho costo de administración sin aportar una ventaja de balance, debido a que las iteraciones ya eran bastante uniformes. Por esta razón la versión final utilizó explícitamente:

`schedule(static)`

También se utilizó `omp_set_dynamic(0)` para evitar que el runtime redujera automáticamente la cantidad de threads solicitada.

3.2. Blurring de imágenes

En Blur no se paralelizó la lectura del archivo ni la escritura del resultado. Estas operaciones avanzan sobre un flujo de archivo compartido, por lo que intentar repartirlas directamente entre threads podría alterar el orden de los bytes.

La parte paralelizada fue el procesamiento de las filas:

```
#pragma omp parallel for schedule(runtime)
for (int y = 0; y < alto; y++) {
    for (int x = 0; x < ancho; x++) {
        /* procesamiento del pixel */
    }
}
```

Cada thread recibió filas diferentes. Todos podían leer de las matrices originales, pero cada thread escribía solamente en posiciones diferentes de las matrices de salida.

Por esta razón no fue necesario utilizar:

- `critical;`

- `atomic;`
- `locks.`

No existía una variable acumuladora global similar a la de Riemann.

3.2.1. Balance de carga

La mayor parte de las filas realiza aproximadamente la misma cantidad de trabajo. Los bordes requieren menos operaciones, pero representan una parte pequeña de la imagen completa.

La versión paralela utilizó:

```
schedule(runtime)
```

Esto permitió que la política pudiera ser definida por el runtime sin volver a compilar el programa. En las ejecuciones generadas por el benchmark no se cambió manualmente la lógica del algoritmo.

4. Metodología de benchmarking

Para evitar que cada integrante ejecutara pruebas diferentes, se utilizó el archivo:

```
benchmark.py
```

El script realizó automáticamente los siguientes pasos:

1. compiló Blur secuencial;
2. compiló Blur paralelo;
3. compiló Riemann secuencial;
4. compiló Riemann paralelo;
5. ejecutó Riemann con los cuatro valores de N ;
6. ejecutó Blur con las tres imágenes;
7. probó 1, 2, 4 y 8 threads;
8. realizó tres repeticiones por configuración;
9. calculó la mediana;

10. calculó speedup y eficiencia;
11. generó archivos CSV, un resumen y gráficas.

Se utilizaron tres repeticiones y se reportó la mediana para reducir el efecto de alguna corrida aislada que pudiera tardar más por procesos del sistema operativo.

El tiempo utilizado por el benchmark se midió desde el exterior del proceso. Esto es importante especialmente para Blur, ya que el tiempo incluye no solamente el filtro, sino también la lectura y escritura del archivo.

Las métricas utilizadas fueron:

$$S(p) = \frac{T_s}{T_p}$$

donde T_s es el tiempo de la versión secuencial y T_p es el tiempo de la versión paralela.

La eficiencia se calculó mediante:

$$E(p) = \frac{S(p)}{p}$$

donde p representa la cantidad de threads.

Un speedup mayor que 1 representa una mejora de tiempo, mientras que un valor menor que 1 significa que la paralelización terminó siendo más lenta que la versión secuencial.

5. Resultados y métricas individuales

Los resultados siguientes no deben compararse únicamente por el tiempo absoluto entre integrantes, ya que fueron obtenidos en equipos diferentes. Lo más útil es observar cómo cambió cada programa respecto a su propia versión secuencial.

5.1. Nery Molina – 23218

5.1.1. Suma de Riemann

Para la carga más grande, $N = 500,000,000$, se obtuvieron los siguientes resultados:

Cuadro 4: Riemann de Nery con $N = 500,000,000$.

Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	0.282386	1.000	1.000
Paralelo – 1 thread	0.279988	1.009	1.009
Paralelo – 2 threads	0.144953	1.948	0.974
Paralelo – 4 threads	0.080137	3.524	0.881
Paralelo – 8 threads	0.055665	5.073	0.634

Se observó una reducción clara del tiempo conforme aumentó la cantidad de threads. Con ocho threads el tiempo bajó aproximadamente de 0,282 s a 0,056 s, obteniendo un speedup de 5,073.

La eficiencia no se mantuvo en 100 %, algo esperado debido al costo de administrar threads, realizar la reducción y compartir los recursos del procesador.

También se notó que el tamaño del problema fue importante. El speedup con ocho threads aumentó de la siguiente forma:

Cuadro 5: Speedup de Nery con ocho threads en Riemann.

N	Speedup	Eficiencia
10,000,000	1.671	0.209
50,000,000	2.611	0.326
100,000,000	3.304	0.413
500,000,000	5.073	0.634

Esto muestra que cuando había más trabajo disponible, el costo fijo de OpenMP se volvió menos significativo.

5.1.2. Blur

Para la imagen `grande.bmp` se obtuvo:

Cuadro 6: Blur de Nery utilizando la imagen grande.

Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	0.190016	1.000	1.000
Paralelo – 1 thread	0.194215	0.978	0.978
Paralelo – 2 threads	0.182496	1.041	0.521
Paralelo – 4 threads	0.183082	1.038	0.259
Paralelo – 8 threads	0.180838	1.051	0.131

La mejora fue bastante pequeña. El mejor valor mostrado en esta prueba fue aproximadamente $1,051\times$, por lo que agregar más threads no produjo una reducción similar a la observada en Riemann.

Esto se relaciona con que el benchmark mide el programa completo. La lectura y escritura de la imagen siguen siendo secuenciales y únicamente la región donde se aplica el filtro utiliza OpenMP.

5.1.3. Evidencia de ejecución de Nery

Las siguientes capturas corresponden a ejecuciones reales de Riemann con $N = 500,000,000$. Estas capturas fueron utilizadas como evidencia visual y no para reemplazar las mediciones del benchmark oficial.

```

dev2retail -- -zsh -- 125x35
Metodo: Secuencial
N: 50000000
Intervalo: [0, 1]
Resultado: 0.333333333333336
Tiempo: 0.77232958 s
dev2retail@Dev2Retail-MacBook-Pro ~ %

```

(a) Ejecución secuencial.

```

dev2retail -- -zsh -- 125x35
Metodo: Paralelo
N: 50000000
Threads: 2
Intervalo: [0, 1]
Resultado: 0.3333333333333276
Tiempo: 0.386348098 s
dev2retail@Dev2Retail-MacBook-Pro ~ %

```

(b) Ejecución con 2 threads.

```

dev2retail -- -zsh -- 125x35
Metodo: Paralelo
N: 50000000
Threads: 4
Intervalo: [0, 1]
Resultado: 0.3333333333333294
Tiempo: 0.1980991 s
dev2retail@Dev2Retail-MacBook-Pro ~ %

```

(c) Ejecución con 4 threads.

```

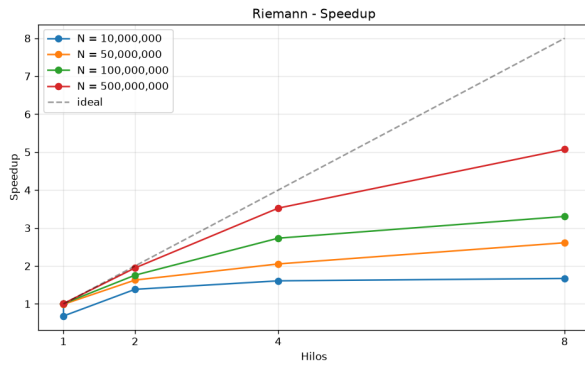
dev2retail -- -zsh -- 125x35
Metodo: Paralelo
N: 50000000
Threads: 8
Intervalo: [0, 1]
Resultado: 0.3333333333333326
Tiempo: 0.119968054 s
dev2retail@Dev2Retail-MacBook-Pro ~ %

```

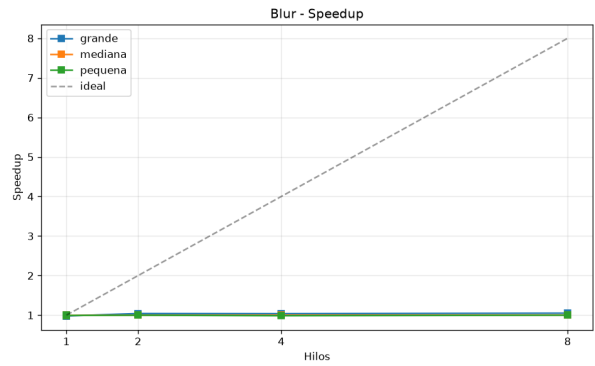
(d) Ejecución con 8 threads.

Figura 1: Evidencia de las ejecuciones realizadas por Nery.

5.1.4. Gráficas generadas en el benchmark de Nery



(a) Speedup de Riemann.



(b) Speedup de Blur.

Figura 2: Resultados generados automáticamente en el benchmark de Nery.

5.2. Renato Rojas – 23813

5.2.1. Suma de Riemann

Para $N = 500,000,000$, Renato obtuvo:

Cuadro 7: Riemann de Renato con $N = 500,000,000$.

Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	0.289919	1.000	1.000
Paralelo – 1 thread	0.281274	1.031	1.031
Paralelo – 2 threads	0.153023	1.895	0.947
Paralelo – 4 threads	0.084085	3.448	0.862
Paralelo – 8 threads	0.055597	5.215	0.652

El patrón fue parecido al obtenido por Nery. Al aumentar el tamaño del problema, ocho threads consiguieron aprovechar mucho mejor el trabajo disponible. Para la carga de 500 millones de subdivisiones se logró un speedup de 5,215.

5.2.2. Blur

Para la imagen grande:

Cuadro 8: Blur de Renato utilizando la imagen grande.

Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	0.213113	1.000	1.000
Paralelo – 1 thread	0.229889	0.927	0.927
Paralelo – 2 threads	0.201993	1.055	0.528
Paralelo – 4 threads	0.204972	1.040	0.260
Paralelo – 8 threads	0.209483	1.017	0.127

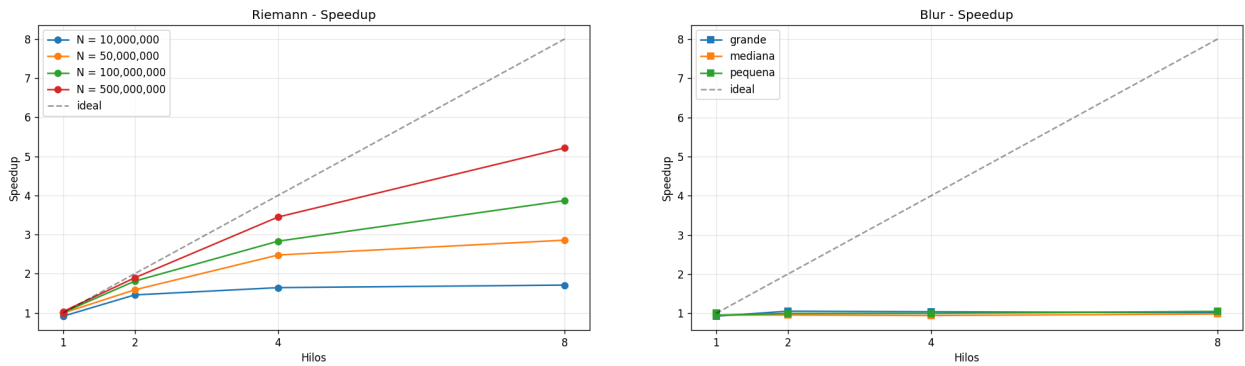
Nuevamente la ganancia fue pequeña. En este caso el mejor resultado de esta tabla apareció con dos threads, con un speedup de aproximadamente 1,055. Al seguir agregando threads la eficiencia disminuyó.

5.2.3. Evidencia de ejecución de Renato

```
ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/paralela/Examen-Parcial-1-Paralela/secuencial/blu
r/src$ ./blurring_secuencial ../../data/blur/grandebmp ../../data/blur/salida_sec_grande.bmp
Error al abrir los archivos. Verifica la ruta: ../../data/blur/grandebmp
ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/paralela/Examen-Parcial-1-Paralela/secuencial/blu
r/src$ ./blurring_secuencial ../../data/blur/grande.bmp ../../data/blur/salida_sec_grande.bmp
Imagen desenfocada con éxito (version secuencial)!
ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/paralela/Examen-Parcial-1-Paralela/secuencial/blu
r/src$ ./blurring_secuencial ../../data/blur/mediana.bmp ../../data/blur/salida_sec_mediana.bmp
Imagen desenfocada con éxito (version secuencial)!
ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/paralela/Examen-Parcial-1-Paralela/secuencial/blu
r/src$ ./blurring_secuencial ../../data/blur/pequena.bmp ../../data/blur/salida_sec_pequena.bmp
Imagen desenfocada con éxito (version secuencial)!
ren@REN:/mnt/c/Users/darta/OneDrive/Desktop/REN/UVG/2026/Segundo ciclo/paralela/Examen-Parcial-1-Paralela/secuencial/blu
```

Figura 3: Captura de ejecución de Blur almacenada por Renato.

5.2.4. Gráficas generadas en el benchmark de Renato



(a) Speedup de Riemann.

(b) Speedup de Blur.

Figura 4: Resultados generados automáticamente en el benchmark de Renato.

5.3. Abby Donis – 22440

5.3.1. Suma de Riemann

Para $N = 500,000,000$, los resultados de Abby fueron:

Cuadro 9: Riemann de Abby con $N = 500,000,000$.

Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	1.664521	1.000	1.000
Paralelo – 1 thread	1.144718	1.454	1.454
Paralelo – 2 threads	0.558781	2.979	1.489
Paralelo – 4 threads	0.325355	5.116	1.279
Paralelo – 8 threads	0.210594	7.904	0.988

En esta computadora se observó un speedup de 7,904 con ocho threads, siendo el valor más alto de las tres corridas para el tamaño grande.

También se obtuvieron eficiencias mayores a 1 en algunas configuraciones. Estos valores no deben interpretarse como que un procesador utilizó más de 100 % de sus recursos. El speedup se calcula respecto a la medición secuencial del mismo equipo y puede verse afectado por cambios de frecuencia del procesador, caché, planificación del sistema operativo y variación entre corridas.

5.3.2. Blur

Para la imagen grande:

Cuadro 10: Blur de Abby utilizando la imagen grande.

Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	0.825080	1.000	1.000
Paralelo – 1 thread	0.757981	1.089	1.089
Paralelo – 2 threads	0.893797	0.923	0.462
Paralelo – 4 threads	0.803147	1.027	0.257
Paralelo – 8 threads	0.876951	0.941	0.118

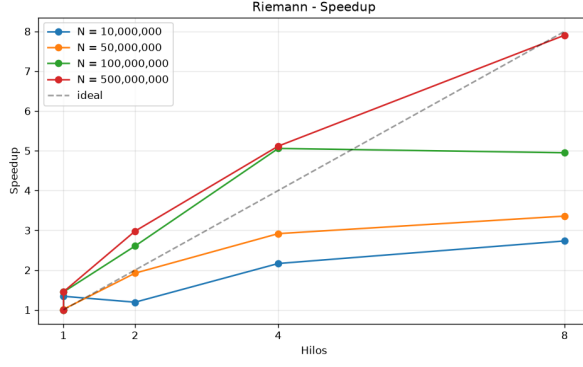
En este caso tampoco se obtuvo una mejora estable conforme aumentaron los threads. Con ocho threads el speedup fue menor que uno, por lo que la versión paralela fue más lenta que la secuencial para esa configuración.

5.3.3. Salida del benchmark de Abby

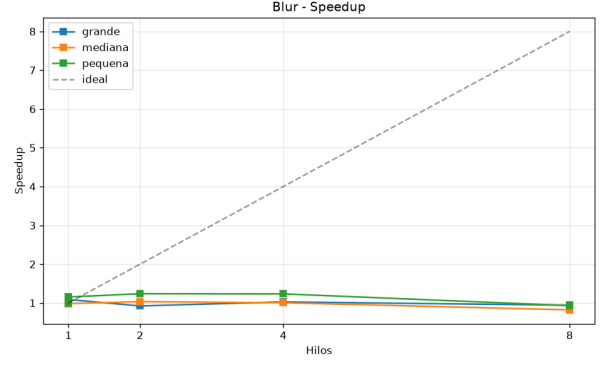
La corrida de Abby quedó almacenada dentro de:

`docs/resultados/resultados_analisis/20260911_203317_Abby/`

Dentro de esta carpeta se encuentran el archivo `tiempos.csv`, `resumen.txt` y las seis gráficas generadas automáticamente.



(a) Speedup de Riemann.



(b) Speedup de Blur.

Figura 5: Gráficas producidas durante el benchmark de Abby.

6. Comparación general

6.1. Riemann

La siguiente tabla resume el caso de $N = 500,000,000$ con ocho threads para los tres integrantes:

Cuadro 11: Comparación individual de Riemann con ocho threads.

Integrante	Sec. (s)	Par. (s)	Speedup	Eficiencia
Nery	0.282386	0.055665	5.073	0.634
Renato	0.289919	0.055597	5.215	0.652
Abby	1.664521	0.210594	7.904	0.988

A pesar de que los tiempos absolutos fueron diferentes por utilizar computadoras distintas, en las tres pruebas se obtuvo la misma tendencia: al aumentar el tamaño de N , Riemann aprovechó mejor los threads.

La carga de 500 millones de subdivisiones fue la que presentó el mejor comportamiento. Esto tiene sentido porque el trabajo realizado por cada iteración es independiente y existe suficiente cantidad de operaciones para compensar el overhead de OpenMP.

6.2. Blur

Para Blur, la comparación de la imagen grande con ocho threads fue:

Cuadro 12: Comparación individual de Blur con ocho threads.

Integrante	Sec. (s)	Par. (s)	Speedup	Eficiencia
Nery	0.190016	0.180838	1.051	0.131
Renato	0.213113	0.209483	1.017	0.127
Abby	0.825080	0.876951	0.941	0.118

Aquí el comportamiento fue muy diferente al de Riemann. Los speedups se mantuvieron cerca de uno y la eficiencia bajó rápidamente al agregar threads.

Esto no significa que OpenMP no haya paralelizado el ciclo. El problema es que el benchmark mide el programa completo, mientras que solamente una parte del flujo de Blur está paralelizada.

De forma general, el programa realiza:

$$T_{\text{total}} = T_{\text{lectura}} + T_{\text{blur}} + T_{\text{escritura}}$$

La lectura y la escritura permanecen secuenciales, así que aunque T_{blur} disminuya, las otras dos partes siguen limitando el tiempo final.

También se observó que la imagen denominada `mediana.bmp` tomó más tiempo que `grande.bmp` en las tres computadoras. Esto muestra que el nombre o el tamaño nominal del archivo no fue suficiente para predecir el costo completo de procesamiento. La representación del BMP, el acceso al archivo y el comportamiento de memoria también influyeron.

7. Resumen de escalabilidad

7.1. Riemann con ocho threads

Cuadro 13: Speedup a ocho threads según el tamaño de Riemann.

N	Nery	Renato	Abby
10,000,000	1.671	1.708	2.732
50,000,000	2.611	2.857	3.357
100,000,000	3.304	3.869	4.949
500,000,000	5.073	5.215	7.904

La tendencia fue consistente. El speedup aumentó junto con N en las tres computadoras.

7.2. Blur con ocho threads

Cuadro 14: Speedup de Blur con ocho threads.

Imagen	Nery	Renato	Abby
Pequeña	0.998	1.050	0.931
Mediana	1.004	0.983	0.822
Grande	1.051	1.017	0.941

En Blur no apareció una relación directa entre utilizar ocho threads y mejorar el tiempo. Dependiendo del integrante y de la imagen, el resultado podía ser ligeramente mejor o incluso peor que la versión secuencial.

8. Limitaciones de las mediciones

Los resultados obtenidos sirvieron para observar el comportamiento real de las dos soluciones, pero existen algunas condiciones que deben considerarse.

Primero, los tres integrantes utilizaron computadoras diferentes. Por esta razón, los tiempos absolutos no se utilizaron para decidir cuál integrante obtuvo “mejores” resultados. El análisis se basó principalmente en speedup y eficiencia dentro de cada computadora.

Segundo, cada configuración fue ejecutada tres veces y se reportó la mediana. Este número fue suficiente para el alcance del parcial, pero una evaluación más grande podría utilizar más repeticiones para reducir todavía más la variación.

Tercero, el benchmark mide el proceso completo. Esto afecta especialmente a Blur, ya que incluye lectura y escritura de archivos que no fueron paralelizadas.

Finalmente, la eficiencia mayor que uno observada en algunas mediciones de Abby debe interpretarse con cuidado. Una corrida secuencial más lenta de lo habitual, cambios de frecuencia del procesador o efectos de caché pueden elevar el speedup calculado. Por esto no se asumió que existiera una aceleración superlineal ideal.

9. Conclusiones

La primera conclusión fue que paralelizar solamente tiene sentido cuando existe suficiente trabajo para justificar el costo adicional. Esto se observó claramente en Riemann. Con los valores pequeños, utilizar threads podía producir mejoras limitadas, pero con $N = 500,000,000$ los tres integrantes obtuvieron una reducción considerable del tiempo.

La segunda conclusión fue que la directiva de reducción fue necesaria en Riemann. Aunque cada iteración podía calcularse independientemente, todas necesitaban contribuir al mismo

acumulador. Utilizar `reduction` evitó una condición de carrera sin obligar a ejecutar cada suma dentro de una región crítica.

También se determinó que `schedule(static)` era una mejor opción para Riemann. Como todas las iteraciones tenían una carga parecida, repartirlas estáticamente evitó el costo extra observado al utilizar scheduling dinámico.

En Blur la situación fue distinta. La región donde se calcula el filtro sí pudo dividirse entre threads sin condiciones de carrera, pero el tiempo total del programa no disminuyó de forma importante. La lectura y escritura de los archivos siguieron siendo secuenciales y representaron una parte considerable de la ejecución. Por esto los speedups se mantuvieron generalmente alrededor de uno.

En general, el parcial permitió comprobar que usar OpenMP no significa automáticamente obtener una mejora. La Suma de Riemann fue un caso donde el paralelismo funcionó bien cuando aumentó la carga, mientras que Blur mostró que una sección paralela puede quedar limitada por otras partes secuenciales del mismo programa.

10. Repositorio y reproducibilidad

Todo el código, imágenes utilizadas, capturas, resultados y gráficas se encuentran en el repositorio:

<https://github.com/Nery2004/Examen-Parcial-1-Paralela>

La estructura principal utilizada fue:

```
Examen-Parcial-1-Paralela/
|
|-- secuencial/
| |-- riemann/
| '-- blur/
|
|-- paralelo/
| |-- riemann/
| '-- blur/
|
|-- data/
| |-- riemann/
| '-- blur/
|
|-- docs/
| |-- resultados/
| |-- screenshots/
| '-- graficas/
|
|-- benchmark.py
```

```
'-- README.md
```

Para volver a ejecutar el benchmark se puede utilizar:

```
python benchmark.py --integrante "Nombre del integrante"
```

En Windows también se puede utilizar:

```
py benchmark.py --integrante "Nombre del integrante"
```

Cada ejecución genera una carpeta independiente dentro de:

```
docs/resultados/resultados_analisis/
```

Las tres corridas utilizadas en este informe fueron:

- 20260911_003210_Renato Rojas
- 20260911_183410_Jose Nery Molina Figueroa
- 20260911_203317_Abby

Cada carpeta contiene el resumen, archivo CSV y las gráficas correspondientes a la corrida individual.

A. Gráficas adicionales

A.1. Nery Molina

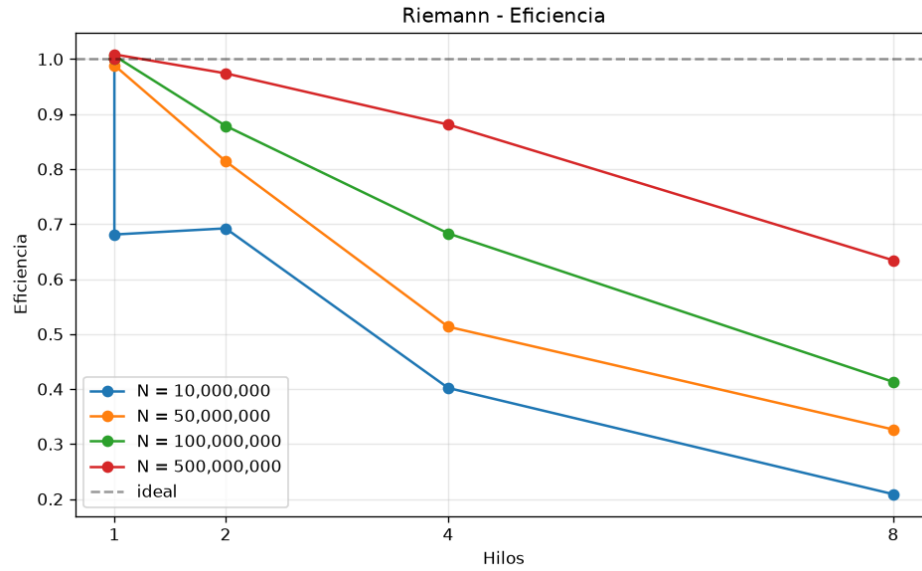


Figura 6: Eficiencia de Riemann obtenida por Nery.

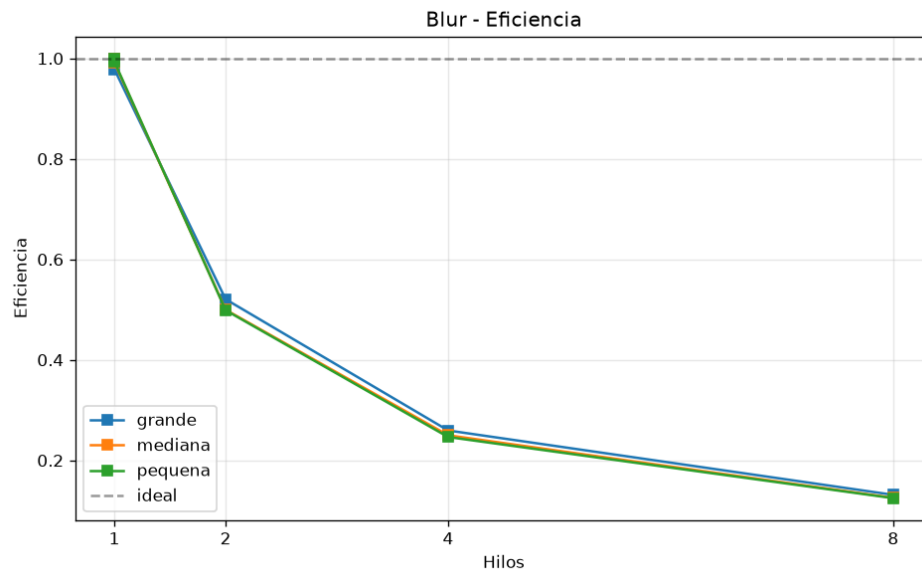


Figura 7: Eficiencia de Blur obtenida por Nery.

A.2. Renato Rojas

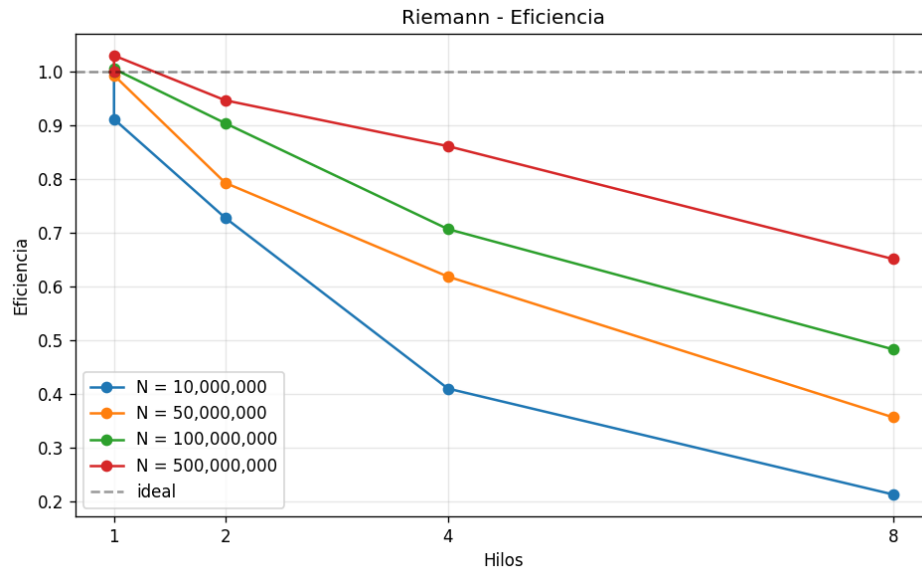


Figura 8: Eficiencia de Riemann obtenida por Renato.

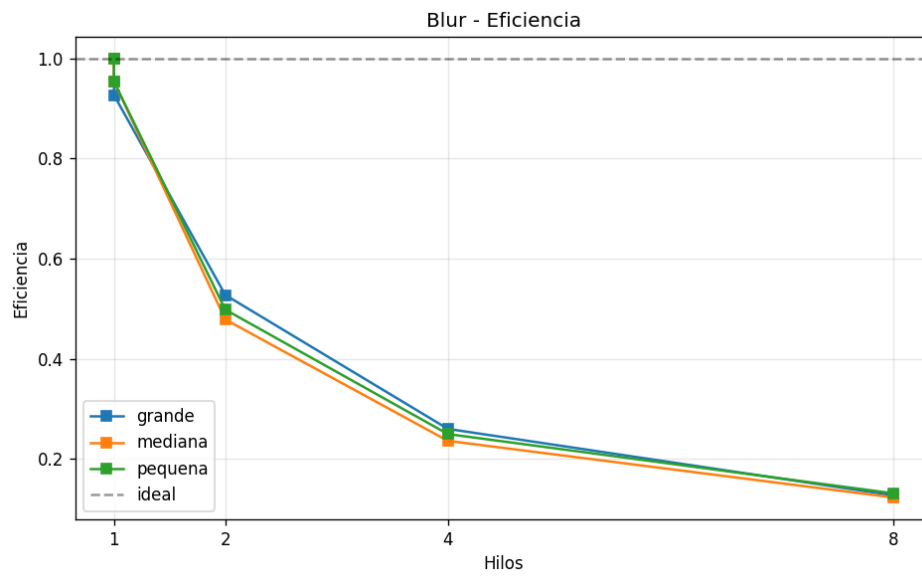


Figura 9: Eficiencia de Blur obtenida por Renato.

A.3. Abby Donis

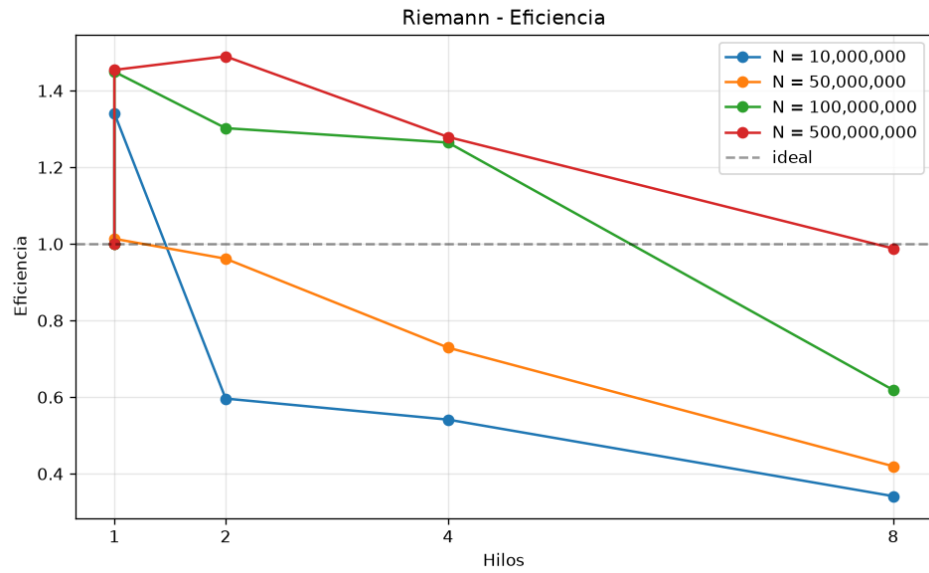


Figura 10: Eficiencia de Riemann obtenida por Abby.

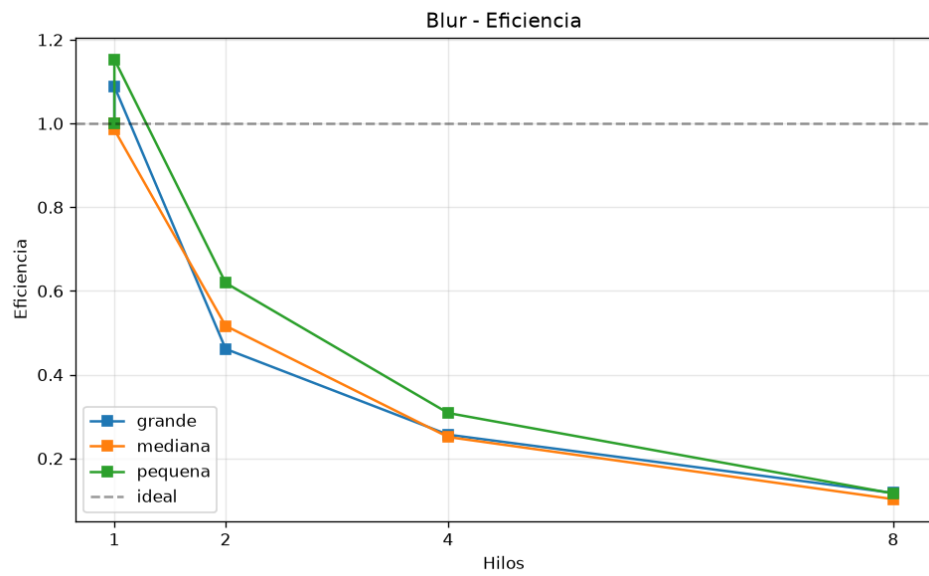


Figura 11: Eficiencia de Blur obtenida por Abby.